INDIA CANDLE AI - TRUSTGRAPH

Final Remediation, Architecture and Submission Report

Version: 10.1.0-final **Date:** 11 Oct 2026 **Challenge:** AfterCode - Missing Update Window + Real-World Feedback

Live: https://india-candle-ai-aftercode.onrender.com/ **GitHub:** https://github.com/suyashshukla911-ops/grevix_india-candle-ai

Executive outcome
The final product is not a repository scanner placed beside a stock dashboard. It is a single trust-aware market-intelligence system where market-data freshness, model validation, risk, and repository freshness jointly determine whether a BUY/SELL result may be treated as trusted.

Decision states
TRUSTED BUY / SELL | HOLD | NO-TRADE | REVALIDATE | DATA UNAVAILABLE

Prepared from the current submission source plus the user-supplied test findings and 54-problem audit PDFs. The report intentionally distinguishes verified code behaviour from third-party-provider and hosted-environment limitations.

1. Product Evolution

Original product: India Candle AI was a short-term stock-analysis system. A user selected an Indian security and timeframe, the system engineered market features, validated a time-series model, and produced price/range estimates plus a directional decision.

Challenge 1: The Missing Update Window introduced a second trust problem. A model result can outlive the software environment that generated it. Source code, feature engineering, dependencies, configuration and security state can change overnight.

Challenge 2: The product must be tested by real people, collect criticism, implement at least one change, and preserve the evidence in GitHub under /feedback/.

Final product principle: “What does the model predict?” is no longer enough. The product also asks “Can I still trust the environment and data state that produced this result?”

2. Final System Architecture

MARKET DATA Yahoo live Verified cache Recorded last session	MODEL Features Chronological validation Calibrated classifier	REPOSITORY Fingerprint Dependencies Security / impact
RISK / REGIME Volatility Liquidity Market regime News context	TRUSTGRAPH 90% signal gate Freshness gates Revalidation policy	AUDIT Model versions Signal journal Accepted baselines

Trust decision: repository current + model gate pass + acceptable market-data freshness + acceptable risk/liquidity context -> BUY/SELL may be trusted. If data/repository freshness fails -> REVALIDATE. If model evidence fails -> NO-TRADE. If liquidity is insufficient -> HOLD.

3. Model Correctness and the 90% Requirement

The final build deliberately avoids the unsupported claim that the model is “90% accurate”. The 90% requirement is implemented as a selective signal-precision gate.

Gate: holdout signal precision >= 90% AND minimum holdout signal count AND Wilson lower confidence bound >= 90%. The threshold is selected on a calibration window and reported metrics come from an untouched holdout window.

Confidence is the calibrated model probability, not an arbitrary value clamped into a favourable range. BUY and SELL are not forced when the evidence does not support the gate.

Backtesting enters at the next bar/session open and exits at the next bar/session close. Transaction-cost and slippage assumptions are included. The report also includes a naive direction baseline and a buy-and-hold comparator.

Regression outputs expose MAE and R2. Stop-loss and take-profit levels are labeled as heuristic paper-analysis controls and are not presented as validated profit guarantees.

4. Market Data Reliability

Yahoo Finance remains the primary provider. The system recognises the real limitation that a listed security can be searchable while Yahoo may not publish the requested intraday interval. The UI therefore separates catalogue availability from interval availability.

Provider path: Yahoo live -> verified cache -> recorded last-session data -> explicit unavailable. The recorded data is real recorded candle data, not synthetic data, and is always freshness-gated before a trusted decision is allowed.

Weekend and NSE-holiday freshness is market-hours aware. While the market is closed, the last completed session can be used for inspection without being mislabeled as an active live market. During an open session, recorded replay is explicitly stale and cannot become a trusted live signal.

The watchlist uses a lightweight quote path. One unavailable symbol does not invalidate all other watchlist rows and the full ML analysis engine is not invoked for each watchlist quote.

5. TrustGraph and Generic GitHub Monitoring

Baseline definition: a cryptographic snapshot of tracked files, component fingerprints, dependency state and accepted findings. It is a version record, not a security certification.

Public GitHub repositories can be cloned into isolated per-session directories, size-limited and count-limited. The repository does not need to be a stock project. It can be an LMS, API, cybersecurity application, web project or another software repository.

The repository workflow is: Clone -> Accept baseline -> Check/Refresh -> Detect drift -> Estimate impact -> Revalidate as needed. External repositories are audit-only until the user explicitly binds them to model trust.

The overnight proof-of-concept operates on the real model workspace and changes real model/configuration/dependency files, rather than a disposable six-file stub.

6. Security, Auth and Abuse Controls

Security scanning uses AST-level checks for eval/exec/os.system/pickle.loads/yaml.load/verify=False and unsafe SQL f-strings, plus credential-like patterns including DB_PASSWORD. Optional Bandit and pip-audit deep scanning is included in runtime requirements.

Repository mutation endpoints require an admin session. The arbitrary local source endpoint is not exposed. GitHub webhooks require an HMAC secret. External clones are isolated, size-limited and cleaned up. Model artifact loading from disk requires a configured HMAC signing key.

An accepted finding is marked as accepted baseline. Only newly introduced HIGH findings block model-bound trust. This avoids the “REVALIDATE forever” failure while preserving the audit history.

7. Challenge 2 - Real-World Feedback

The final submission contains a /feedback/ folder with the real collected evidence and feedback record. The product loop is: real user -> criticism -> specific usability problem -> actual UI/product change -> before/after proof.

The targeted feedback theme is usability of TrustGraph/Repository Health and the explanation of why an environment change can turn a prediction into REVALIDATE. Additional evidence covers chart readability, risk explanation and user expectations around stock search/coverage.

8. 54-Problem Remediation Matrix

#	Finding	Status	Remediation
1	Wrong precision/coverage/threshold semantics	FIXED	Named threshold result; holdout gate uses distinct fields.
2	1d timeframe crash	FIXED	Daily session filtering removed; daily warm-up policy.
3	Weekend/holiday stale logic	FIXED	Market-hours and NSE-holiday aware freshness.
4	Synthetic fallback could look real	FIXED	Synthetic fallback disabled; recorded session is explicit.
5	Accepted HIGH finding blocks forever	FIXED	Only new HIGH findings block.
6	Datetime round warning	FIXED	Fingerprint rounds numeric market columns only.
7	.env not scanned	FIXED	Exact .env included.
8	DB_PASSWORD missed	FIXED	Credential pattern expanded.
9	Random-walk false gate pass	FIXED	Minimum signals + Wilson lower bound.
10	Weak 90% claim	FIXED	Holdout precision and lower bound are reported.
11	Five-signal minimum	FIXED	30 intraday / 15 daily defaults.
12	Validation leakage	FIXED	Fit/calibration/untouched holdout.
13	Heuristic confidence	FIXED	Calibrated probability.
14	No honest regression metrics	FIXED	MAE/R2 added.
15	Entry at signal close	FIXED	Next bar/session open entry.
16	No trading costs	FIXED	Cost/slippage assumptions included.
17	Buy-and-hold gap error	FIXED	Price-window comparator.
18	No naive baseline / sizing	FIXED	Naive baseline + 1x paper sizing + explicit heuristic risk note.

#	Finding	Status	Remediation
19	Zero return counted as down	FIXED	Flat moves neutral.
20	SELL ambiguity	FIXED	SELL/EXIT is explicitly exit-oriented paper action.
21	Yahoo cloud reliability	MITIGATED	Live/cache/recorded-session/unavailable layers.
22	Unadjusted prices	FIXED	auto_adjust=True.
23	Intraday history limits	MITIGATED	Provider limits respected; availability explicit.
24	Index data unused	FIXED	Nifty/BankNifty/VIX context features.
25	Unvalidated news wordlist	IMPROVED	Timestamped, age-decayed context; bounded trust influence.
26	Illiquid SME signals	FIXED	Minimum value-traded floor.
27	Brittle catalogue scraping	MITIGATED	Bundled seed + refresh.
28	Monitoring only a stub	FIXED	Actual app repository is model workspace.
29	Overnight simulation only stub	FIXED	Real model/config/dependency files changed.
30	Scanner false positives	FIXED	AST targeted security checks.
31	Scanner dangerous misses	FIXED	AST coverage expanded.
32	Bandit/pip-audit absent	FIXED	Included and enabled for deep scans.
33	Package-version mismatch risk	FIXED	Package versions in environment fingerprint/model metadata.
34	Accepted baseline security semantics	FIXED	Accepted vs blocking finding state.
35	Public mutation routes	FIXED	Admin authentication.
36	Arbitrary source/local route	FIXED	Removed.

#	Finding	Status	Remediation
37	Shared repository state	FIXED	Browser session isolation.
38	Broken webhook	FIXED	HMAC verification + real GitHub refresh.
39	Unbounded clones	FIXED	Size/count limits.
40	Old clones accumulate	FIXED	Automatic cleanup.
41	Untrusted joblib load	MITIGATED	Signed-artifact load boundary.
42	Global analysis lock	FIXED	Per session/symbol/timeframe lock.
43	Retrain every candle	FIXED	Refresh cadence + environment invalidation.
44	Duplicate journal/no scoring	FIXED	Unique keys + later-candle settlement.
45	Render free disk ephemeral	DOCUMENTED	Durable storage recommended for production.
46	Incomplete packaging	FIXED	Tests, requirements, Dockerfile, render.yaml, env example.
47	Doc/test count drift	FIXED	Unified docs and explicit 27-test suite.
48	Loose deps/Python mismatch	FIXED	Pinned requirements + Python 3.12.11.
49	Dashboard overloaded	IMPROVED	Primary market workflow + separated TrustGraph/Repository tabs.
50	Public BUY/SELL compliance	FIXED	Paper-analysis/no-orders disclaimers and caution.
51	Legacy fabricated confidence	LEGACY ARCHIVED	Legacy code not used by final runtime.
52	Legacy sentiment shift bug	LEGACY ARCHIVED	Final news service is timestamped/age-decayed.
53	Legacy price leakage concern	LEGACY ARCHIVED	Legacy predictor removed from runtime path.
54	Legacy missing deps	FIXED	Final dependency set covers runtime; legacy is archive only.

9. Test and Packaging Status

Automated suite: 27 pytest tests exist in the final source tree. A full 27/27 run was completed during the V3 hardening cycle before the last documentation/package cleanup. The last hardening changes were syntax-compiled in this report-generation environment. The sandbox cannot re-run the Windows virtual environment because it is a Windows PE environment and outbound PyPI access is blocked.

The final source includes tests for market data isolation, catalogue search, repository lifecycle, security scanning, context-feature merging, market-closed freshness, the 1d path, model fingerprinting, watchlist quote isolation, API contracts and model gate semantics.

Final package checks performed after hardening: Python byte-compilation across app/tests and JavaScript syntax validation of the browser bundle.

The final release excludes .git, .venv, runtime state and pytest caches from the submission archive. The preserved legacy code is stored only under archive/legacy_original/.

10. Render Deployment

The complete application requires a FastAPI Web Service because market data, machine-learning inference and repository auditing are server-side operations. `render.yaml` is included with the production start command and /api/health health check.

Python 3.12.11 is pinned for deployment compatibility. The free Render filesystem is ephemeral, so runtime SQLite/audit/model state should be moved to persistent storage for a production service. The competition deployment remains self-contained and reproducible.

Live demo currently used by the submission: https://india-candle-ai-aftercode.onrender.com/

11. Judge Demonstration Flow

1. Search TCS, choose 5m, and run Analyze Stock. Show current price, chart, validation, risk and explainability.

2. Explain that the 90% number is a selective signal-precision gate, not a blanket prediction-accuracy claim.

3. Open Repository Health. Explain that it can monitor any public GitHub repository.

4. Clone a public repository and show the external-audit scope.

5. For the model environment, accept a baseline -> simulate overnight update -> show modified files/dependency change -> STALE -> revalidation -> accept new baseline -> CURRENT.

6. Return to Market Predictor and show that repository/data/model trust conditions determine whether a directional action can be trusted.

12. Real-World Boundaries

Yahoo Finance is an unofficial third-party source and can be unavailable or incomplete for specific securities/timeframes or from some hosted environments. The application handles this explicitly rather than inventing prices.

Recorded last-session data exists for reproducible operation when live data is unavailable; it is never represented as current live market data when the market is open. No model can guarantee future returns. India Candle AI is a decision-support and paper-analysis prototype; public use of investment research/signals may create regulatory requirements depending on the jurisdiction and service model.